RAPPORT DE PROJET

Système de gestion d'un entrepôt logistique

Base de données relationnelle et application web de gestion

Auteurs	Jad ABDUL RAZEK, Matthias LUTZ, Abdelmalik TELITEL
Enseignant	Alexandre Lombard
Date	Novembre 2026

Table des matières

1. Introduction 3

2. Analyse : cas d'utilisation 4

3. Conception de la base de données 7

- 3.1 Modèle conceptuel 7
- 3.2 Modèle logique 12
- 3.3 Normalisation 14
- 3.4 Données factices 14

4. Automatisation interne à la base 15

5. Sécurité et confidentialité 16

6. Performances 17

7. Fonctionnalités réalisées 18

- 7.1 Authentification et profils 18
- 7.2 Référentiels 19
- 7.3 Quais et rendez-vous 20
- 7.4 Flux de camions 22
- 7.5 Réception des marchandises 24
- 7.6 Emplacements et stocks 26
- 7.7 Préparation et expédition 28
- 7.8 Traçabilité et indicateurs 31
- 7.9 Choix de périmètre sur les suppressions 32

8. Conclusion 33

1. Introduction

Ce projet consiste à concevoir et développer un système de base de données relationnelle pour les opérations quotidiennes d'un entrepôt logistique : flux de camions, quais, transporteurs, réception, stockage et expédition. Le système doit assurer une traçabilité complète des marchandises et fournir des indicateurs de pilotage.

Nous avons réalisé une application web en PHP 8 avec PDO, sur une base MariaDB 10.4. La logique métier critique est placée dans la base : les règles s'appliquent ainsi quel que soit le chemin d'écriture, y compris lors de la génération des données factices.

La démarche a suivi les étapes vues en cours : analyse par les cas d'utilisation, modèle conceptuel, modèle relationnel, implémentation, puis mesure des performances. Le détail des besoins figure dans le cahier des charges, fourni à part.

2. Analyse : cas d'utilisation

Le diagramme `uml/use_case_entrepot.svg` décrit le système selon les cinq acteurs du sujet : administrateur, responsable d'exploitation, agent d'accueil, magasinier / préparateur et transporteur. Ils héritent tous de l'acteur générique « Utilisateur », qui se connecte ; le « Visiteur » peut seulement s'inscrire. Le diagramme compte 47 cas d'utilisation, regroupés selon les étapes du sujet, avec leurs relations `«include»` et `«extend»` (les extensions portent leur condition).

Choix des acteurs. Les acteurs sont exactement les profils cités par le sujet. Le transporteur est un acteur parce que le sujet lui donne un « accès limité à la prise de rendez-vous ». Les clients propriétaires, les fournisseurs et les chauffeurs apparaissent dans le sujet comme des informations à gérer, et non comme des utilisateurs du système. Ce sont donc des données, sans compte ni acteur. La généralisation vers « Utilisateur » évite de relier « Se connecter » aux cinq acteurs, et elle exprime que tous partagent le même mécanisme d'authentification.

Choix des relations.

- Un `«include»` marque une étape toujours exécutée par le cas de base. Par exemple, la prise de rendez-vous vérifie toujours le créneau et la compatibilité véhicule / quai, et la génération des listes applique toujours la règle FIFO / FEFO.
- Un `«extend»` marque un comportement optionnel, et sa condition est notée sur la relation. Par exemple, une réserve n'est émise que si un écart est constaté, et un quai n'est mis hors service qu'en cas de panne ou de maintenance.

Périmètre. Chaque cas d'utilisation correspond à une phrase du sujet. Nous n'avons ajouté aucun acteur ni aucune fonctionnalité que le sujet ne demande pas. Les indicateurs ne sont pas modélisés comme des données : ils sont calculés à partir des données existantes.

Quatre cas phares sont détaillés dans `fiches_cas_utilisation.md` : prendre un rendez-vous, accueillir un camion sans rendez-vous, enregistrer la réception et prélever les articles d'une liste.

Alignement avec le modèle conceptuel. Les deux diagrammes ont été construits ensemble et vérifiés dans les deux sens :

- chaque cas d'utilisation s'appuie sur des entités et des associations du modèle conceptuel ;
- inversement, chacune des 33 entités et des 46 associations sert au moins un cas d'utilisation, ce qui garantit qu'aucun élément du modèle ne déborde du périmètre.

La table de correspondance complète figure à la fin de `fiches_cas_utilisation.md`. En voici le résumé par domaine :

Domaine du sujet	Cas d'utilisation	Éléments du modèle conceptuel
Authentification	S'inscrire, Se connecter, Gérer les comptes	Utilisateur, Profil ; Avoir, Représenter
Transporteurs, chauffeurs, véhicules	Gérer les transporteurs, chauffeurs, véhicules	Transporteur, Contact, Contrat, Chauffeur, Habilitation, Véhicule, Type de véhicule
Quais et rendez-vous	Décrire les quais, Prendre / modifier un RDV, Suivre le statut	Quai, Équipement, Rendez-vous ; Accepter, Équiper, Réserver, Prévoir
Flux de camions	Arrivée, sans RDV, contrôle, parking, appel à quai, (dé)chargement, départ	Passage, Type d'étape ; Honorer, Utiliser, Conduire, Accoster, Franchir

Domaine du sujet	Cas d'utilisation	Éléments du modèle conceptuel
Réception	Avis, réception, écarts, réserve, rangement	Fournisseur, Avis d'expédition, Réception, Lot, Type d'écart, Réserve ; Annoncer, Recevoir, **Constater**
Emplacements et stocks	Structure, catalogue, transfert, ajustement, consultation	Zone, Allée, Rack, Emplacement, Article, Client, Condition de stockage, Mouvement de stock ; Stocker
Préparation et expédition	Commandes, listes, prélèvement, colis, chargement, documents	Commande, Liste de prélèvement, Unité d'expédition, Expédition, Document de transport ; **Prélever**, Contenir, Charger
Traçabilité et indicateurs	Retracer un lot, Consulter les indicateurs, Générer un rapport	Lecture des éléments ci-dessus, sans entité propre

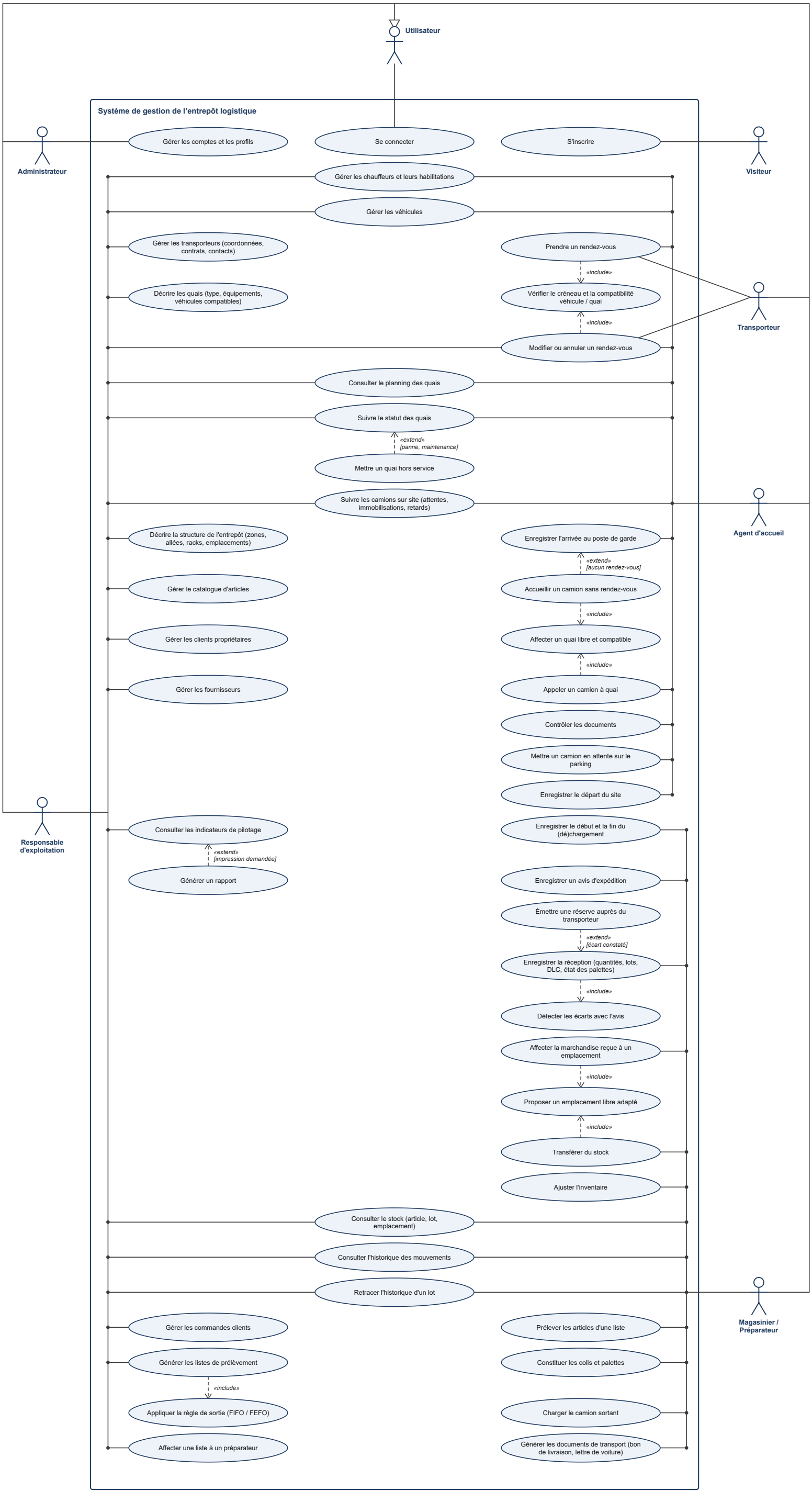


Figure 1 — Diagramme de cas d'utilisation

3. Conception de la base de données

3.1 Modèle conceptuel

Le diagramme entité-association (`uml/diagramme_entite_association.svg`) compte 33 entités et 46 associations :

- 33 associations 1:N ;
- 1 association 0..1 / 0..1 (**Honorer**) ;
- 12 associations N:N, dont 2 ternaires.

Notation

- **Formes.** Les entités sont des rectangles, les associations des losanges et les attributs des ellipses. L'identifiant de chaque entité est souligné. Ce n'est jamais un numéro technique : c'est un identifiant métier (SIRET, immatriculation, numéro de permis, code d'emplacement, numéro SSCC…).
- **Sens des flèches.** La flèche donne le sens de lecture du verbe de l'association. Par exemple, `Utilisateur → Avoir → Profil` se lit « un utilisateur a un profil », et `Transporteur → Employer → Chauffeur` se lit « un transporteur emploie des chauffeurs ».
- **Lecture des cardinalités.** Une cardinalité placée du côté d'une entité indique combien d'occurrences de cette entité sont liées à une occurrence de l'entité opposée.
  - Avec `Utilisateur 0..N — Avoir — 1 Profil`, un utilisateur a exactement un profil, et un même profil est attribué à 0 ou plusieurs utilisateurs.
  - Pour une association ternaire, la cardinalité placée près d'une entité indique combien d'occurrences de cette entité sont liées à un couple d'occurrences des deux autres.
- **Attributs portés par une association.** Une association N:N peut porter des attributs. C'est le cas lorsque la donnée dépend du couple et non de l'une des deux entités : la quantité commandée dépend de la commande *et* de l'article, et la date d'une étape dépend du passage *et* du type d'étape.

Démarche

Nous avons relu le sujet phrase par phrase pour en tirer les entités et les associations :

- un nom qui a une identité propre et des propriétés devient une entité (quai, lot, rendez-vous…) ;
- une propriété simple devient un attribut (DLC, immatriculation…) ;
- un verbe qui relie deux notions devient une association (un chauffeur *détient* une habilitation, un fournisseur *émet* un avis…).

Nous n'avons gardé que ce que le sujet demande ou ce qui est nécessaire pour produire un résultat qu'il exige. Par exemple, l'adresse de livraison d'une commande est nécessaire pour éditer le bon de livraison. Les indicateurs (taux d'occupation, temps d'attente, taux d'écart, productivité…) ne sont pas stockés : ils se calculent à partir des passages, des étapes horodatées, des constats et des listes de prélèvement.

Choix de modélisation par domaine

Utilisateurs et profils.

- Le profil est une **entité** et non une simple valeur, car il porte un code, un libellé et une description, et l'administrateur peut le consulter.
- Un utilisateur a exactement un profil, car le sujet parle de profils « aux droits distincts ».

- L'association **Représenter** (Utilisateur 0..N — 0..1 Transporteur) relie un compte transporteur à sa société. C'est ce lien qui limite le transporteur à ses propres rendez-vous et véhicules. Les autres profils ne sont rattachés à aucun transporteur, d'où la cardinalité minimale 0.

Transporteurs, chauffeurs, véhicules.

- **Contact** et **Contrat** sont des entités distinctes du transporteur, car une société a plusieurs contacts et plusieurs contrats successifs, chacun avec ses propres propriétés. Les placer en attributs du transporteur violerait la 1NF.
- **Habilitation** est une entité reliée au chauffeur par une N:N, **Détenir**, qui porte les dates d'obtention et d'expiration. Une habilitation (matières dangereuses, par exemple) est partagée par plusieurs chauffeurs, mais sa date d'expiration est propre à chaque chauffeur.
- **Type de véhicule** (tracteur, remorque, porteur, fourgon) est une entité. Elle sert de pivot à la compatibilité avec les quais (voir ci-dessous) : une simple liste de valeurs ne permettrait pas de l'associer aux quais.
- Les caractéristiques frigorifique et hayon restent des attributs booléens du véhicule. Ce sont des propriétés oui / non, sans données propres.

Quais et rendez-vous.

- **Équiper** (Quai — Équipement) et **Accepter** (Quai — Type de véhicule) sont des N:N : un quai a plusieurs équipements et accepte plusieurs types de véhicules, et inversement.
- La compatibilité véhicule / quai demandée par le sujet se vérifie par le chemin `Véhicule → Être de type → Type de véhicule ← Accepter ← Quai`.
- Le **Rendez-vous** est une entité, et non une association entre Quai et Véhicule. Il possède son propre identifiant, un créneau, un sens et un statut, et un même véhicule peut avoir plusieurs rendez-vous sur le même quai à des dates différentes. Une association Quai — Véhicule n'autoriserait qu'un seul lien par couple.
- L'absence de chevauchement est une règle de gestion, et non une structure. Elle est garantie par un déclencheur (section 4).

Flux de camions.

- Le **Passage** (la visite réelle d'un camion) est distinct du **Rendez-vous** (la visite prévue). Le sujet demande de « détecter les retards par rapport au rendez-vous prévu », ce qui suppose de conserver les deux.
- L'association **Honorer** est en 0..1 des deux côtés : un rendez-vous peut ne jamais être honoré, et un passage peut exister sans rendez-vous, comme le demande le sujet avec les arrivées sans rendez-vous.
- Les étapes (arrivée, contrôle des documents, parking, appel à quai, début et fin d'opération, départ) ne sont pas sept colonnes de dates dans Passage. Elles sont modélisées par une entité **Type d'étape** et une N:N **Franchir**, qui porte l'horodatage. Ce choix évite des colonnes vides tant que le camion n'a pas franchi l'étape, et l'attribut *Ordre* du type d'étape permet de contrôler la succession des étapes. Les temps d'attente et d'immobilisation se calculent alors par différence entre deux horodatages.
- Le quai réellement utilisé (**Accoster**, 0..1) est distinct du quai réservé : il n'est connu qu'après l'appel à quai, et il peut différer du quai réservé.

Réception.

- **Annoncer** (Avis d'expédition — Article) porte la quantité attendue, et **Recevoir** (Réception — Lot) porte la quantité reçue et l'état des palettes. Les écarts se calculent en comparant ces deux associations.
- **Correspondre à** est en 0..1 côté avis, car une livraison peut arriver sans avoir été annoncée.

- **Lot** est une entité distincte d'**Article**. Le numéro de lot et la DLC sont propres à chaque lot, et c'est au niveau du lot que s'appliquent la traçabilité et la règle FEFO.
- La **Réserve** est une entité rattachée à la réception (association **Porter sur**). Elle est émise auprès du transporteur pour la livraison dans son ensemble, et elle a son propre numéro, sa date et l'information « transmise ». Le transporteur concerné se retrouve par la réception, le passage puis le véhicule : le dupliquer dans la réserve créerait une redondance.

Emplacements et stocks.

- La structure physique suit la hiérarchie du sujet : **Zone** → **Allée** → **Rack** → **Emplacement**, avec trois associations 1:N. Chaque niveau a son propre code et ses propres liens : c'est la zone qui autorise des conditions de stockage, et c'est le rack qui porte le nombre de niveaux. Un emplacement appartient à un seul rack, ce qui situe sans ambiguïté son allée et sa zone.
- Le *niveau* est un attribut de l'emplacement et non une entité, car il n'a aucune donnée propre.
- La règle de compatibilité produit / zone est portée par l'entité **Condition de stockage** : un article *nécessite* une condition (ambiant, froid, dangereux…) et une zone *autorise* une ou plusieurs conditions. Un emplacement est compatible si sa zone autorise la condition de l'article.
- **Stocker** (Emplacement — Lot, avec la quantité) représente l'état du stock « par article, lot et emplacement » demandé par le sujet. L'article n'y figure pas, car il se déduit du lot.
- Le **Mouvement de stock** est une entité et non une association. Il a son propre numéro, une date, un motif et un auteur (**Effectuer**). Il relie un lot à deux emplacements optionnels, **Provenir de** et **Aller vers**, tous deux en 0..1. Ces deux cardinalités suffisent à représenter les quatre types de mouvements du sujet :

Type de mouvement	Emplacement d'origine	Emplacement de destination
Entrée	aucun	oui
Sortie	oui	aucun
Transfert	oui	oui
Ajustement d'inventaire	l'un ou l'autre, selon le sens	l'un ou l'autre, selon le sens

L'historisation des mouvements est demandée explicitement par le sujet. Elle est rendue immuable dans la base (section 4), et la cohérence entre le type du mouvement et ses emplacements est garantie par une contrainte `CHECK`.

Commandes, préparation, expédition.

- **Comprendre** (Commande — Article) porte la quantité commandée. Ce sont les lignes de commande du sujet.
- Une **Liste de prélèvement** sert une seule commande (**Servir**). Elle est affectée à 0..1 préparateur (**Préparer**), car elle existe dès sa génération, avant d'être affectée. Ses dates de début et de fin servent à mesurer la productivité des préparateurs.
- L'**Unité d'expédition** (colis ou palette) appartient à une commande (**Constituer**) et contient des lots (**Contenir**, avec la quantité). Elle est ensuite chargée dans une **Expédition** (**Charger**), elle-même emportée par un passage de camion sortant (**Enlever**). Ce chemin permet de répondre à la demande de traçabilité du sujet : « réception, emplacements successifs, commande et camion d'expédition » d'un lot.
- Le **Document de transport** est une entité typée (bon de livraison ou lettre de voiture) qui accompagne une expédition. Les deux documents ont la même structure, ce qui ne justifie pas deux entités distinctes.

Transporteur, fournisseur et client. Ces trois entités ont des attributs semblables (SIRET, raison sociale, adresse). Nous ne les avons pas regroupées en une entité générique « Tiers » :

- elles ne participent à aucune association commune ;
- elles jouent des rôles différents : le transporteur emploie et possède, le fournisseur émet des avis, le client possède des articles et passe des commandes ;
- un même SIRET peut légitimement figurer dans deux rôles.

Une généralisation aurait alourdi le modèle sans éliminer de vraie redondance.

Associations ternaires

Avant de décomposer une association portant sur trois entités en associations binaires, nous avons appliqué un critère : une association reste ternaire lorsqu'aucun couple d'entités ne détermine la troisième, et que ses attributs dépendent des trois à la fois. Deux faits du sujet vérifient ce critère.

- **Constater** (Réception, Article, Type d'écart), qui porte la quantité et la description.
  - Le sujet demande d'enregistrer « les écarts (manquants, excédents, marchandises endommagées) ». Un écart est donc le constat qu'un article d'une réception présente un certain type d'écart, pour une certaine quantité.
  - La réception et l'article ne suffisent pas : un même article peut être à la fois manquant et endommagé.
  - La réception et le type d'écart ne suffisent pas non plus : plusieurs articles d'une réception peuvent être manquants.
  - Enfin, l'article et le type d'écart se répètent d'une réception à l'autre.
  - La quantité dépend des trois. Une décomposition en binaires perdrait cette information.
- **Prélever** (Liste de prélèvement, Lot, Emplacement), qui porte la quantité à prélever, la quantité prélevée et l'ordre de passage.
  - Une ligne de prélèvement indique au préparateur de prendre tel lot à tel emplacement pour telle liste.
  - Un même lot peut être réparti sur plusieurs emplacements, et une liste peut puiser le même lot à plusieurs endroits. Ni « liste + lot » ni « liste + emplacement » ne suffit à identifier la ligne.
  - Le choix du lot (FIFO / FEFO) et celui de l'emplacement se font ensemble.

Pour chacune de ces deux ternaires, les trois branches sont en 0..N.

Nous avons à l'inverse gardé en binaire plusieurs associations où une troisième entité semblait pouvoir s'ajouter, car celle-ci s'en déduit déjà :

- **Stocker** relie un emplacement et un lot. Ajouter l'article serait redondant, puisque Lot → Article est une dépendance fonctionnelle, et cela violerait la 3NF.
- **Contenir** relie une unité d'expédition et un lot. La commande est déjà déterminée par l'unité (**Constituer**).
- **Recevoir** relie une réception et un lot. Le fournisseur et le passage se déduisent de la réception, et l'emplacement de rangement relève d'un mouvement de stock, qui est un évènement distinct.

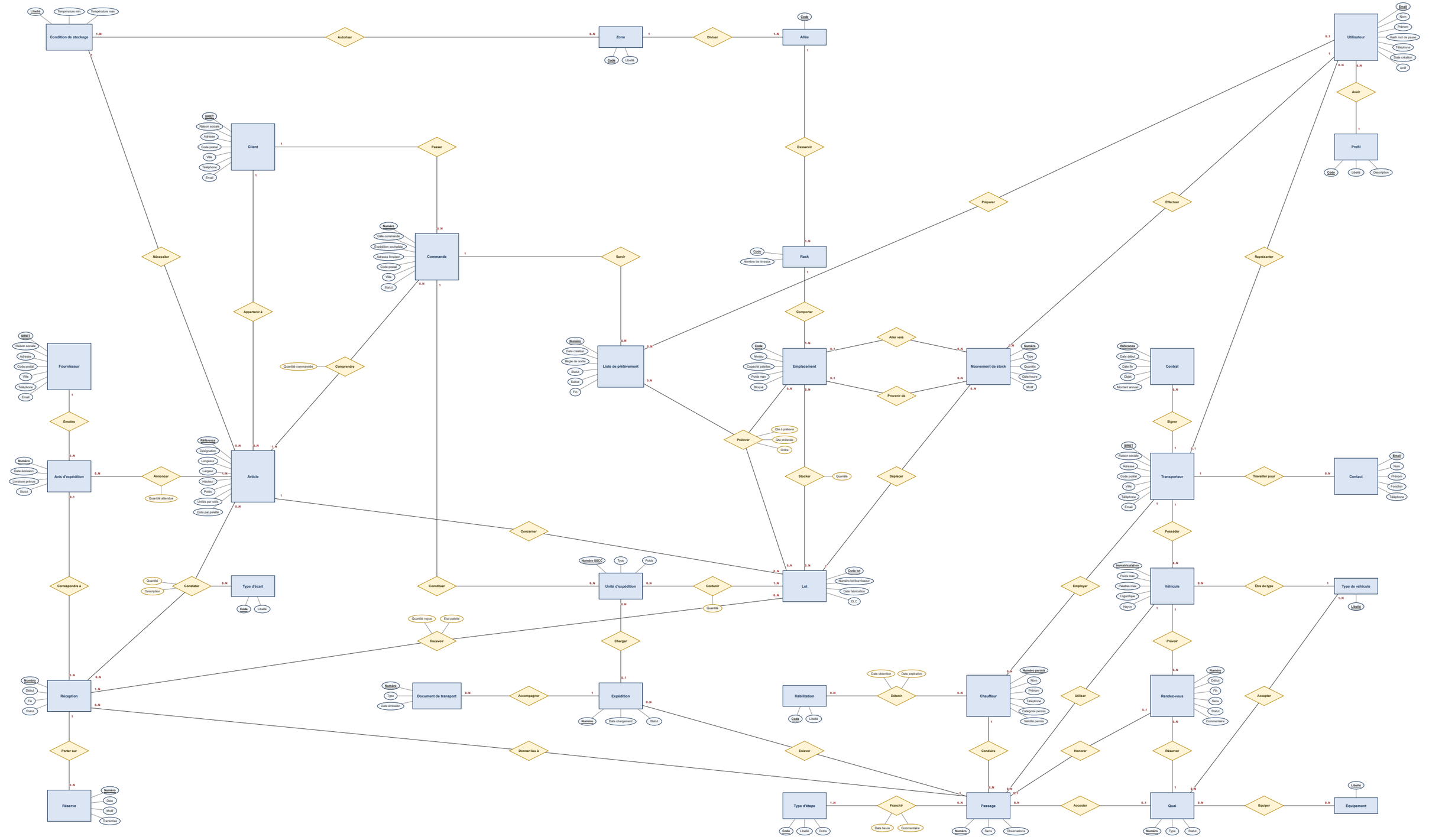


Figure 2 — Diagramme entité-association (modèle conceptuel de données)

3.2 Modèle logique

Le modèle relationnel compte 45 tables : les 33 entités et les associations N:N ou ternaires devenues tables de jointure. Le schéma `uml/schema_relationnel.svg` le représente : clés primaires soulignées, clés étrangères préfixées par `#` et reliées par une flèche à la table référencée, valeurs uniques en italique. Le détail table par table figure dans `schema_relationnel.md`.

Principes retenus :

- Chaque table a une clé primaire artificielle `id`, sauf les tables de jointure, dont la clé est composée des clés étrangères.
- Les identifiants métier (SIRET, immatriculation, numéro de rendez-vous…) sont déclarés `UNIQUE`. Ce sont eux qui sont affichés, jamais les `id`.
- Les contraintes `CHECK` protègent les valeurs : formats de SIRET et de code postal, dates cohérentes, quantités positives, cohérence source / destination d'un mouvement.
- Plusieurs associations N:N portent des attributs, par exemple `passage_etape` (horodatage) ou `ligne_commande` (quantité). Les deux ternaires deviennent `constat_ecart` (clé : réception, article, type d'écart) et `ligne_prelevement` (liste, lot, emplacement).

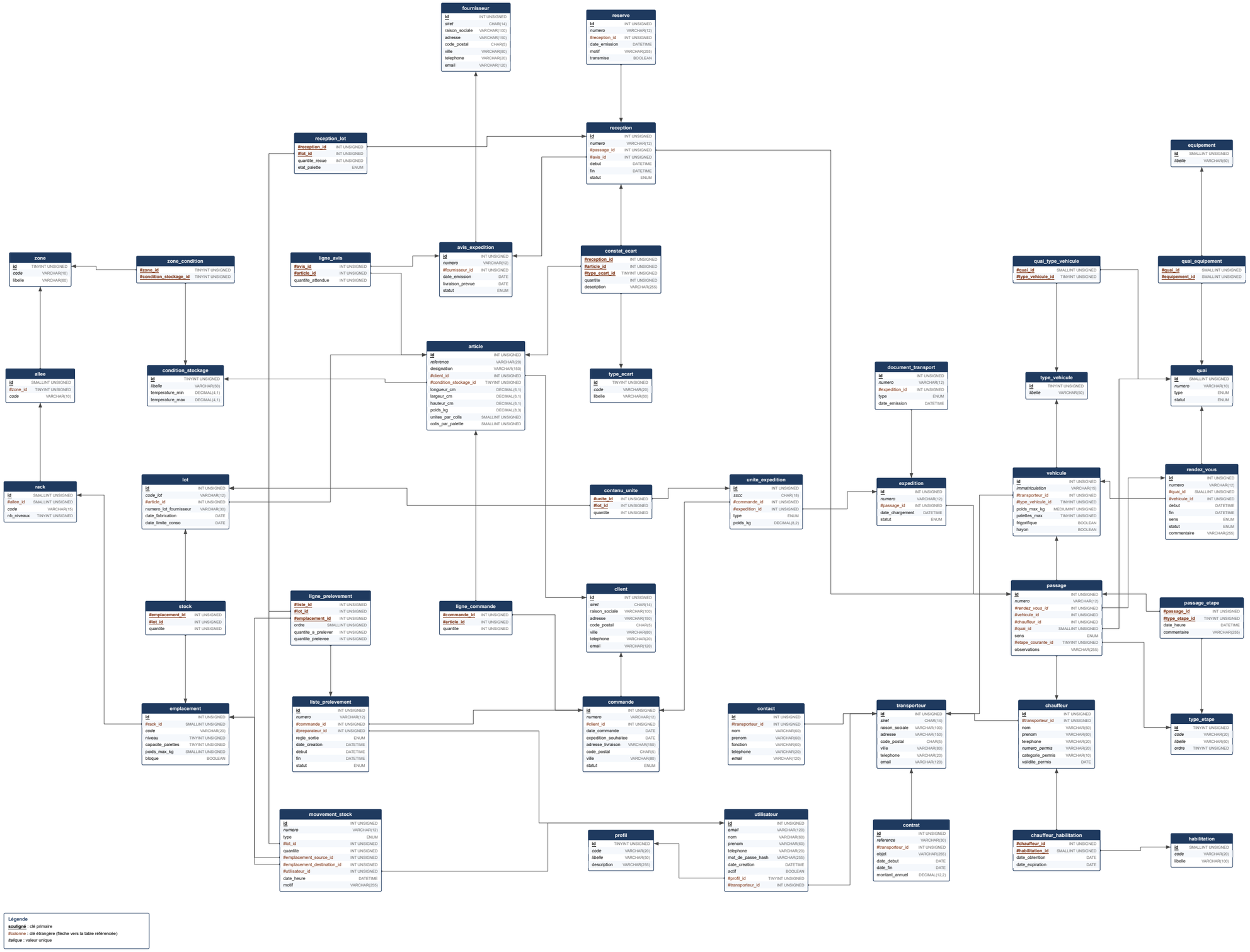


Figure 3 — Schéma relationnel (modèle logique de données)

3.3 Normalisation

La base respecte la 1NF : toutes les valeurs sont atomiques et les listes (équipements d'un quai, habilitations d'un chauffeur) sont des tables de jointure. Elle respecte la 2NF : dans les tables à clé composée, les attributs dépendent de toute la clé. Elle respecte la 3NF, sous réserve des choix ci-dessous, tous délibérés.

Donnée	Pourquoi elle pourrait sembler redondante	Justification
`stock.quantite`	Se déduit de la somme des mouvements	Synthèse maintenue exclusivement par déclencheur, pour éviter de sommer tout l'historique. L'application ne peut pas y écrire
`passage.etape_courante_id`	Se déduit de `passage_etape`	Évite une sous-requête dans chaque écran temps réel. Maintenue par déclencheur
`quai.statut`	« occupé » se déduit des passages	Maintenu par déclencheur. « Hors service » est une saisie, et « réservé », dépendant de l'heure, reste calculé par la vue `v_quai_temps_reel`
`passage.vehicule_id`	Présent aussi sur le rendez-vous	Le passage décrit le véhicule réel, le rendez-vous le véhicule prévu. Un passage peut aussi exister sans rendez-vous
`passage.sens`	Présent aussi sur le rendez-vous	Même raison : une arrivée sans rendez-vous doit porter son sens
`commande.adresse_livraison`	Ressemble à l'adresse du client	L'adresse de livraison est propre à la commande et peut différer du siège
`liste_prelevement.statut`	Lié à `debut` et `fin`	Statut explicite du traitement, lisible et filtrable ; la contrainte `CHECK` assure la cohérence des dates
`reception.statut`	Lié à `fin`	Une date de fin peut être saisie avant la clôture, qui reste une action distincte déclenchant le calcul des écarts
`code_postal` et `ville`	Le code postal semble déterminer la ville	Ce n'est pas une dépendance fonctionnelle : un code postal peut couvrir plusieurs communes

3.4 Données factices

`sql/installer.php` crée la base puis simule environ deux mois d'activité, soit environ 120 000 lignes, en passant par les déclencheurs et procédures. Les données respectent donc les règles métier : après installation, le stock correspond exactement à la somme des mouvements et aucun emplacement ne dépasse sa capacité.

4. Automatisation interne à la base

Mécanisme	Nombre	Rôle
Déclencheurs	20	Numérotation, contrôles métier (créneaux, étapes, capacité des emplacements et des camions), mise à jour du stock et des statuts, immuabilité des mouvements
Procédures	6	Contrôle d'un rendez-vous, contrôle du chargement d'un camion, clôture de réception, génération et clôture de listes, proposition d'emplacements
Fonctions	4	Palettes occupées, palettes et poids libres, date d'une étape
Vues	5	Quais en temps réel, durées des passages, détail du stock, remplissage, dates limites

Historisation. La table `mouvement_stock` est immuable : deux déclencheurs refusent toute modification ou suppression, et une correction passe par un ajustement. Chaque mouvement garde sa date, son utilisateur et son motif, et chaque passage garde l'horodatage de toutes ses étapes : l'historique d'un lot se reconstitue de sa réception jusqu'au camion qui l'a emporté.

5. Sécurité et confidentialité

- **Mots de passe** : hachés avec `password_hash` (bcrypt), jamais stockés en clair.
- **Injection SQL** : toutes les valeurs saisies passent par des requêtes préparées.
- **XSS** : chaque donnée affichée est échappée par la fonction `e()`.
- **CSRF** : tout formulaire porte un jeton vérifié à la réception.
- **Droits** : chaque page contrôle le profil côté serveur. Un transporteur ne voit que ses propres rendez-vous, et un compte désactivé perd son accès à la page suivante.
- **Moindre privilège** : le site se connecte avec le compte `entrepot_app`, qui ne peut ni modifier la structure ni écrire dans `stock`, tenu uniquement par les déclencheurs.
- **Erreurs** : les erreurs techniques sont écrites dans le journal d'erreurs PHP et jamais affichées.

Limites connues : le site n'impose pas HTTPS et ne bloque pas les comptes après plusieurs échecs de connexion (il ralentit seulement chaque tentative). Un déploiement réel devrait traiter ces deux points.

6. Performances

Les requêtes principales sont mesurées par `sql/performances.php` ; le détail figure dans `rapport_performances.md`. Toutes s'exécutent en quelques millisecondes sur le jeu de données fourni, à l'exception de la proposition d'emplacements (quelques dizaines de millisecondes), acceptable pour un écran de rangement.

Les index les plus utiles sont l'index composite `ix_rdv_quai_creneau`, qui rend le contrôle de chevauchement environ 14 fois plus rapide, et `ix_mvt_lot_date`, qui rend l'historique d'un lot environ 7 fois plus rapide.

7. Fonctionnalités réalisées

7.1 Authentification et profils

Objectif. Permettre au personnel de s'inscrire et de se connecter, avec des droits distincts.

Travail réalisé. Inscription avec contrôle des saisies, validation par l'administrateur, connexion, cinq profils et menu adapté à chacun. L'administrateur active, modifie, réinitialise et supprime les comptes (`inscription.php`, `connexion.php`, `utilisateurs.php`).

Conclusion. L'usage multi-utilisateurs est complet. L'administrateur est volontairement exclu de l'exploitation.

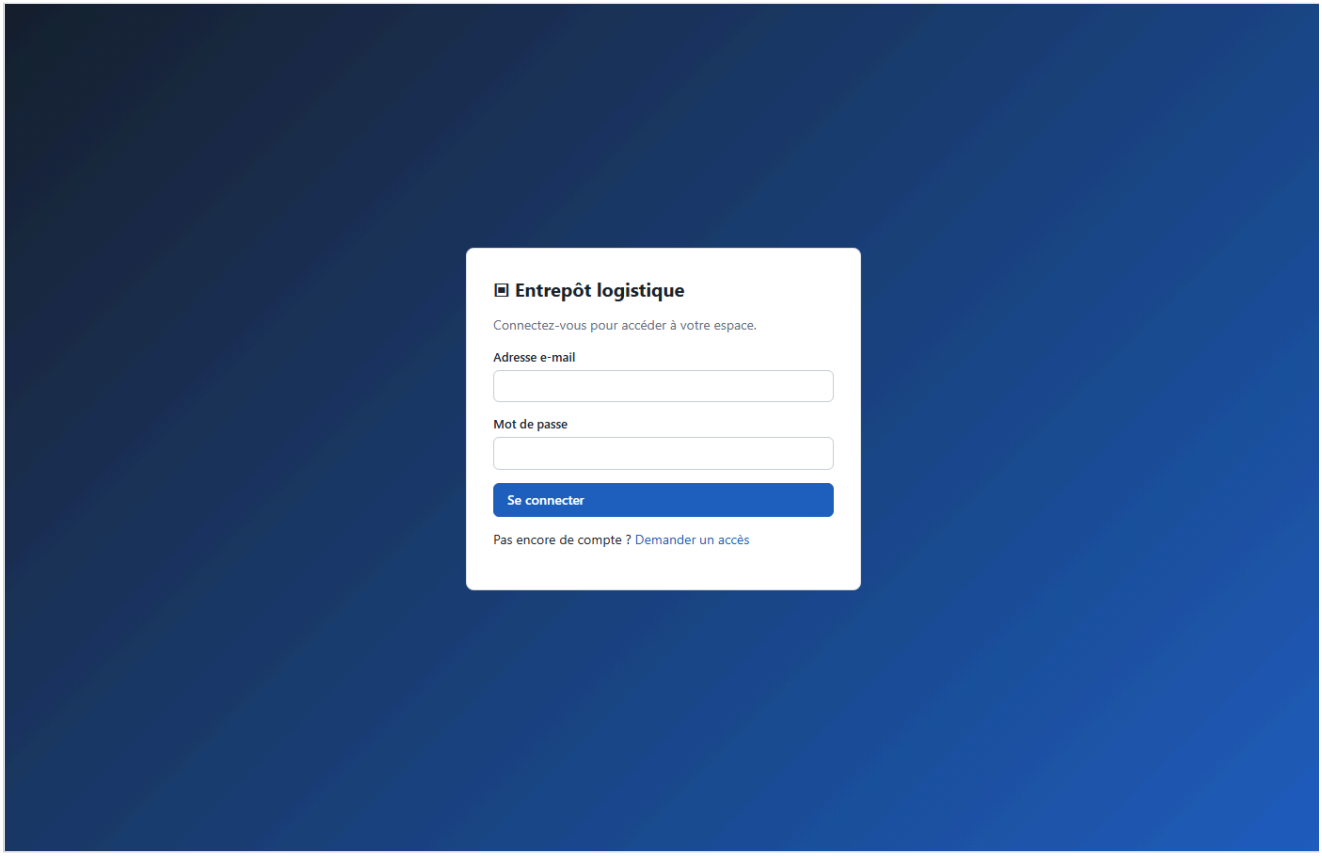


Figure 4 — Page de connexion

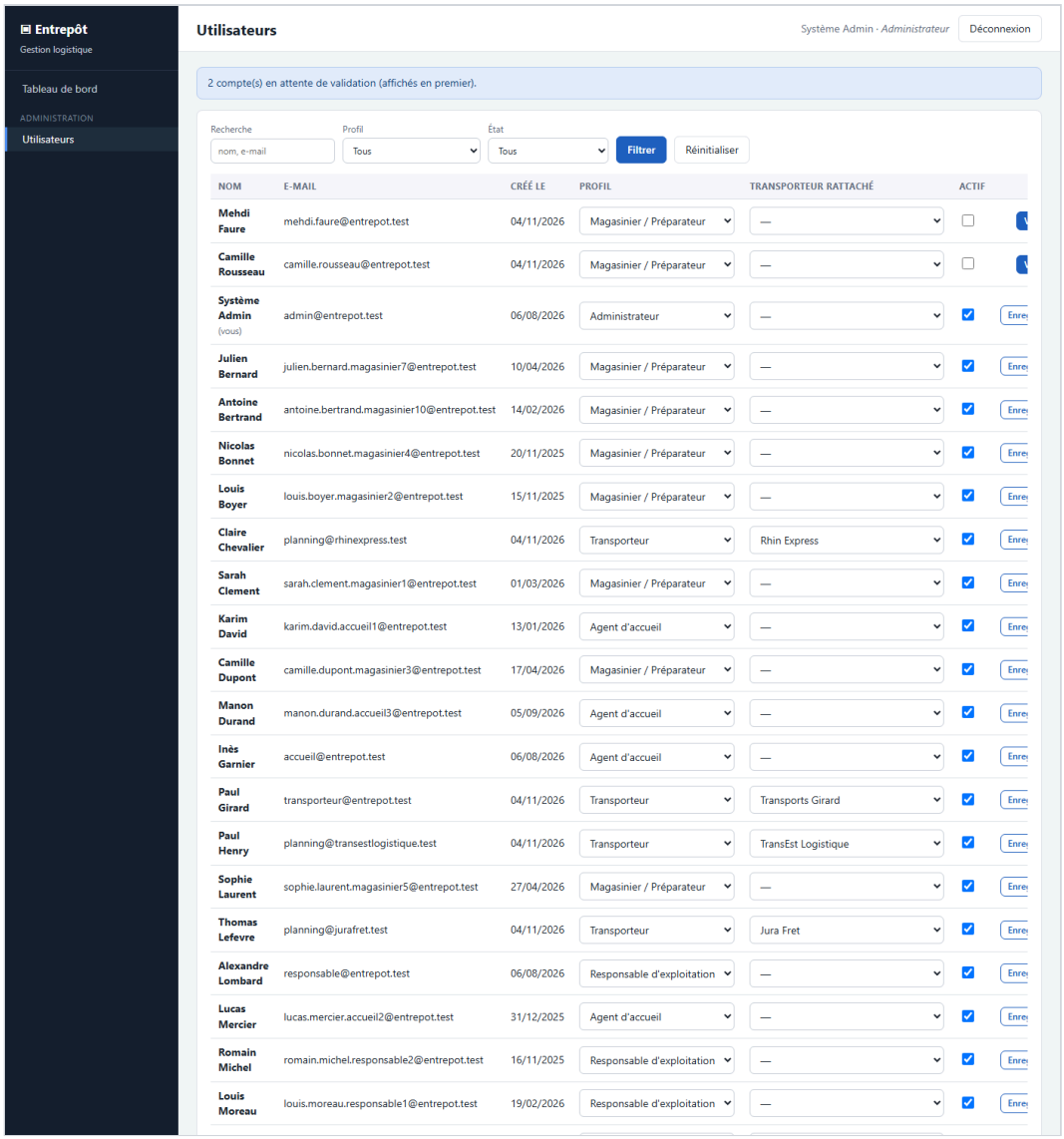


Figure 5 — Gestion des comptes par l'administrateur

7.2 Référentiels

Objectif. Maintenir les transporteurs, chauffeurs, véhicules, clients, fournisseurs, articles et la structure de l'entrepôt.

Travail réalisé. Un écran générique `referentiel.php`, piloté par `includes/referentiels.php`, offre liste, recherche, filtres, ajout, modification et suppression, avec contrôle des saisies.

Conclusion. Cet écran unique assure un comportement homogène. Un élément encore utilisé ne peut pas être supprimé, et l'utilisateur en est informé.

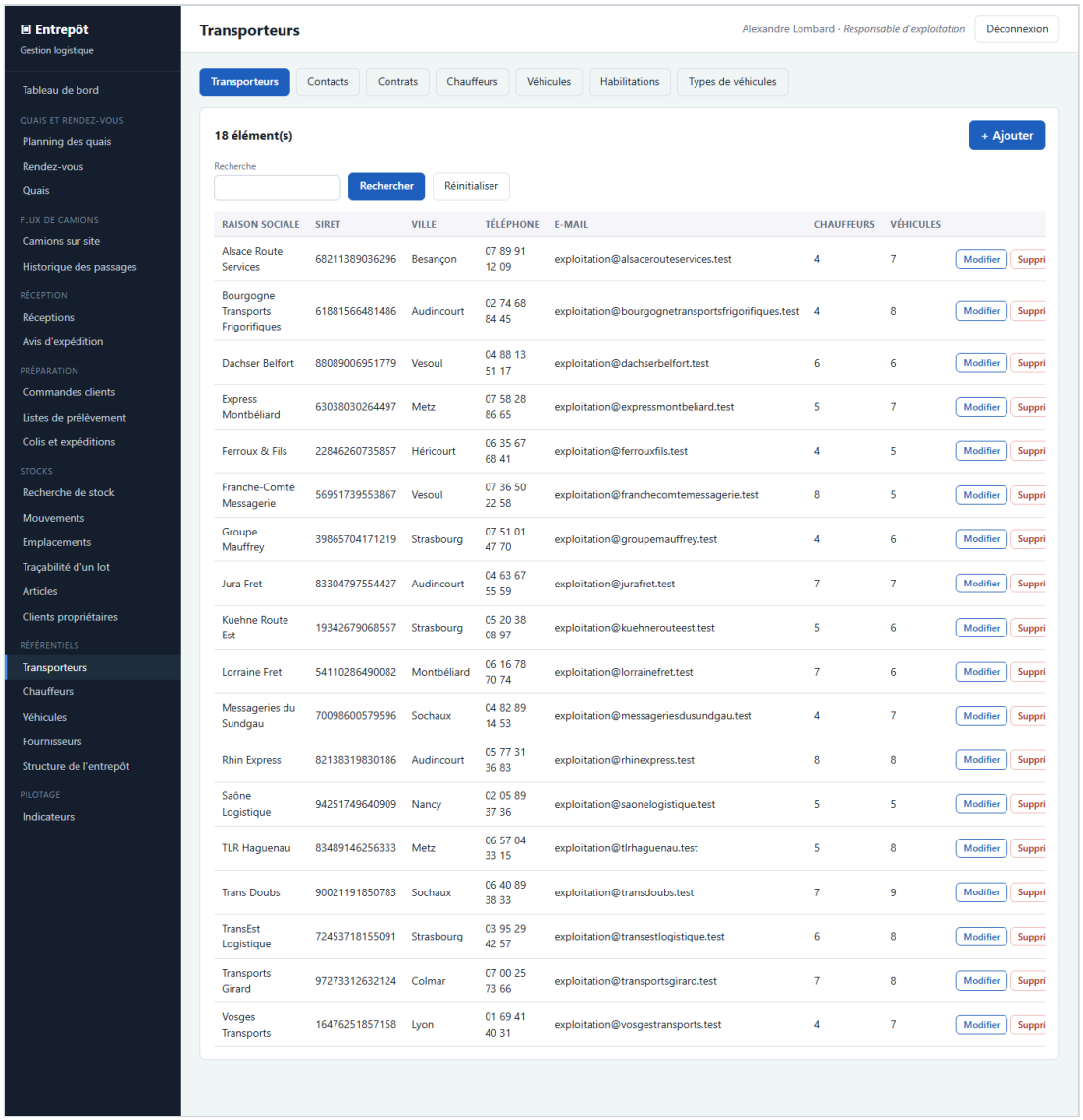


Figure 6 — Écran générique des référentiels (transporteurs) : recherche, filtres et actions

7.3 Quais et rendez-vous

Objectif. Planifier les quais sans chevauchement et suivre leur statut.

Travail réalisé. Planning journalier, prise et modification de rendez-vous avec recherche des quais compatibles et libres, recherche multicritère, export CSV, gestion des quais et de leur statut (`planning.php`, `rdv.php`, `rdv_form.php`, `quais.php`).

Conclusion. Les règles sont vérifiées par déclencheur : le chevauchement est impossible, même hors de l'interface. La mise hors service d'un quai signale les rendez-vous à déplacer, sans les déplacer automatiquement.

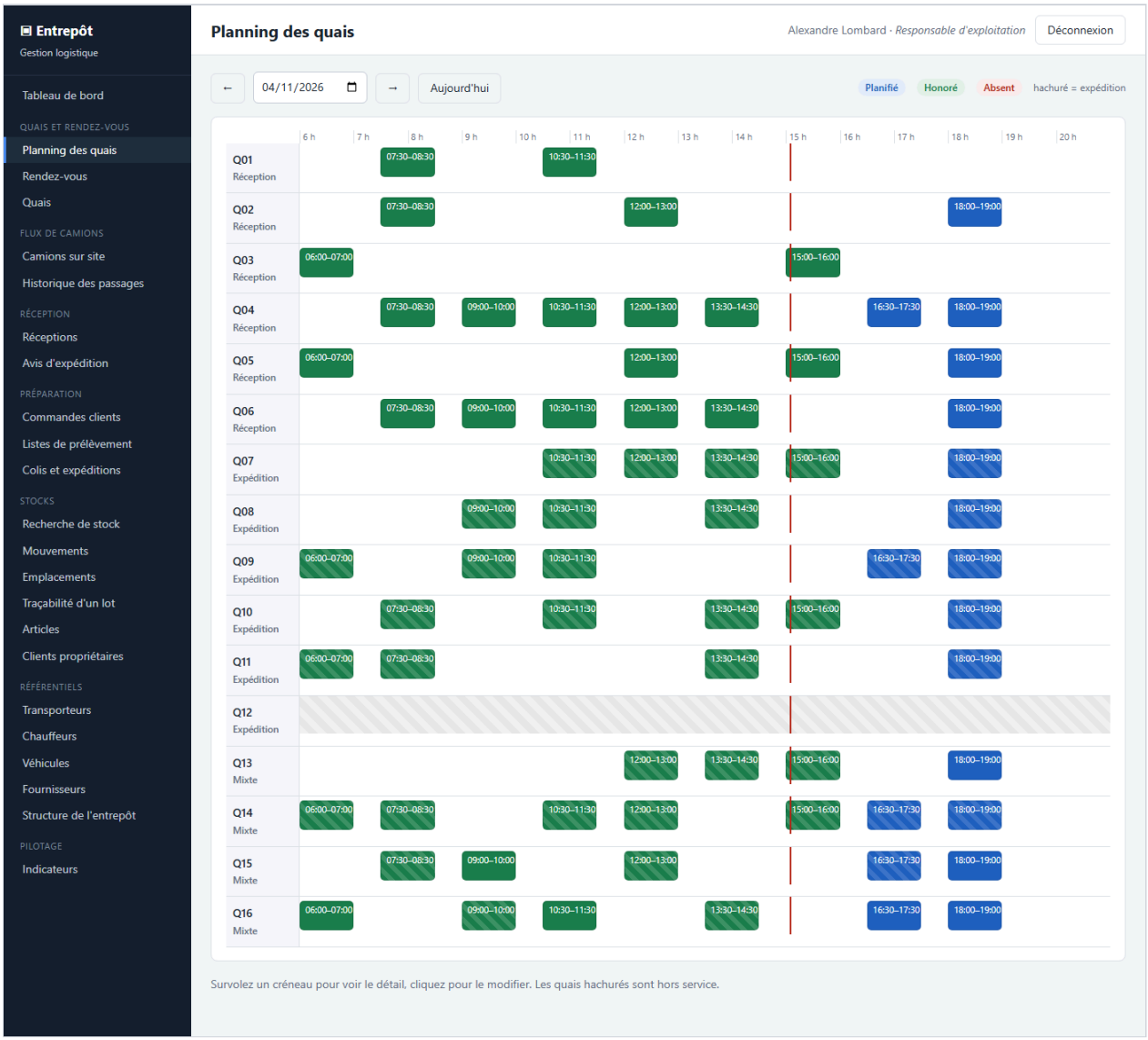


Figure 7 — Planning journalier des quais

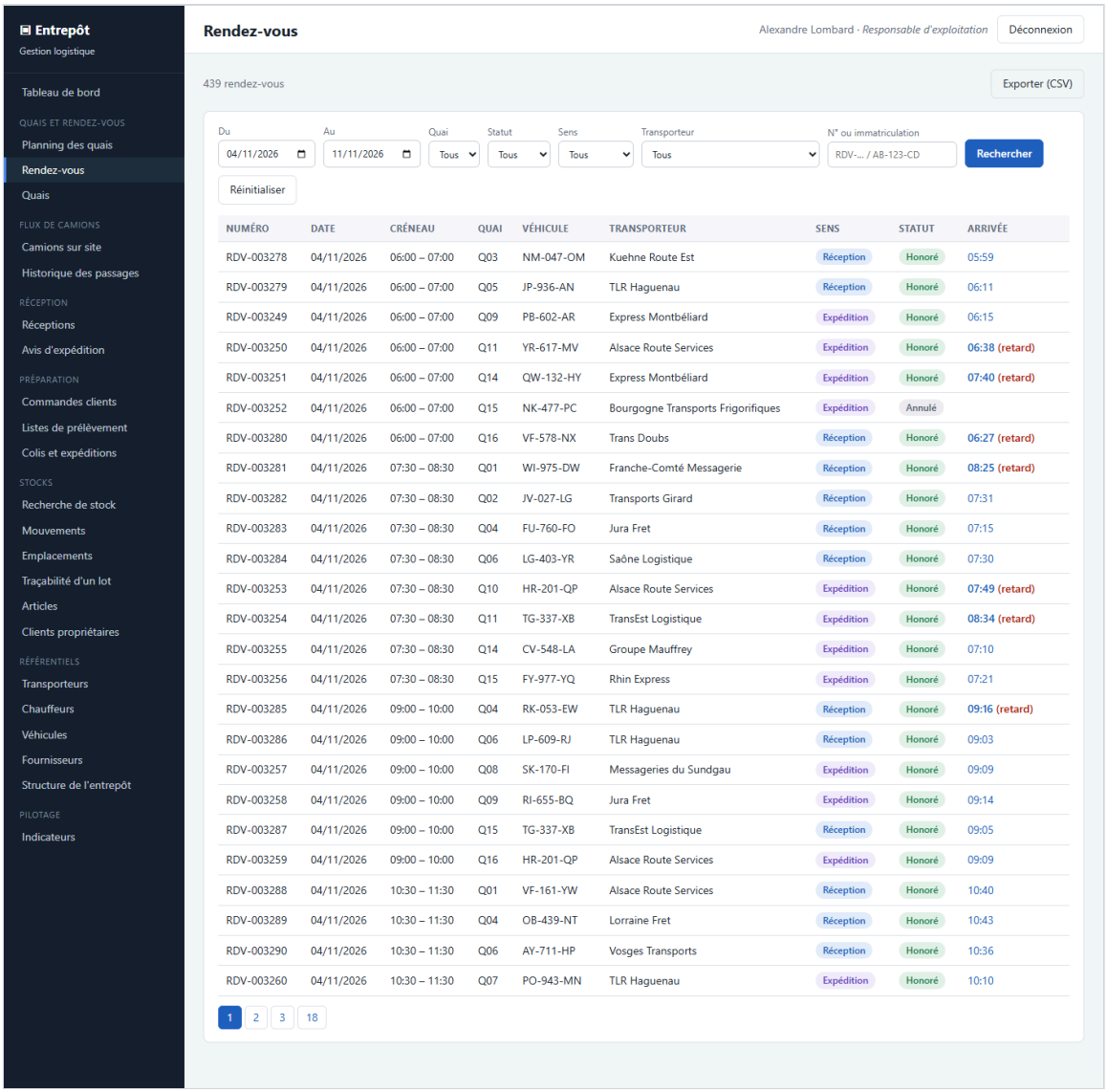


Figure 8 — Liste des rendez-vous avec recherche multicritère

7.4 Flux de camions

Objectif. Horodater chaque étape du passage d'un camion et mesurer les temps.

Travail réalisé. Tableau des camions sur site avec passage d'étape en un clic, arrivées avec ou sans rendez-vous, proposition d'un quai libre et compatible, historique avec durées (`flux.php`, `passage_arrivee.php`, `passage.php`, `passages.php`).

Conclusion. L'ordre des étapes, la disponibilité du quai et sa compatibilité sont garantis par la base : un camion ne peut pas commencer son (dé)chargement sans avoir été appelé à quai.

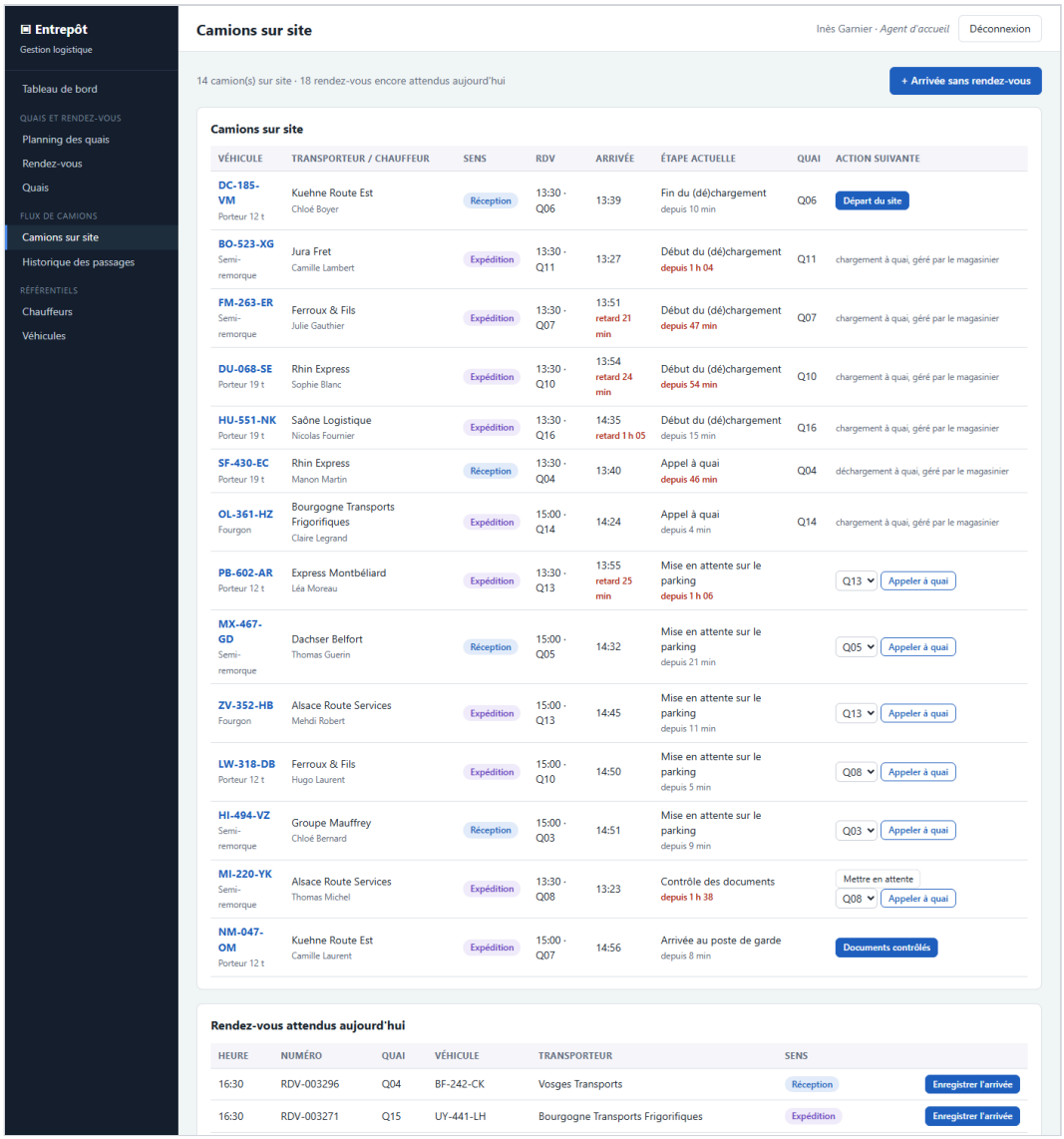


Figure 9 — Camions sur site et action suivante pour chaque passage

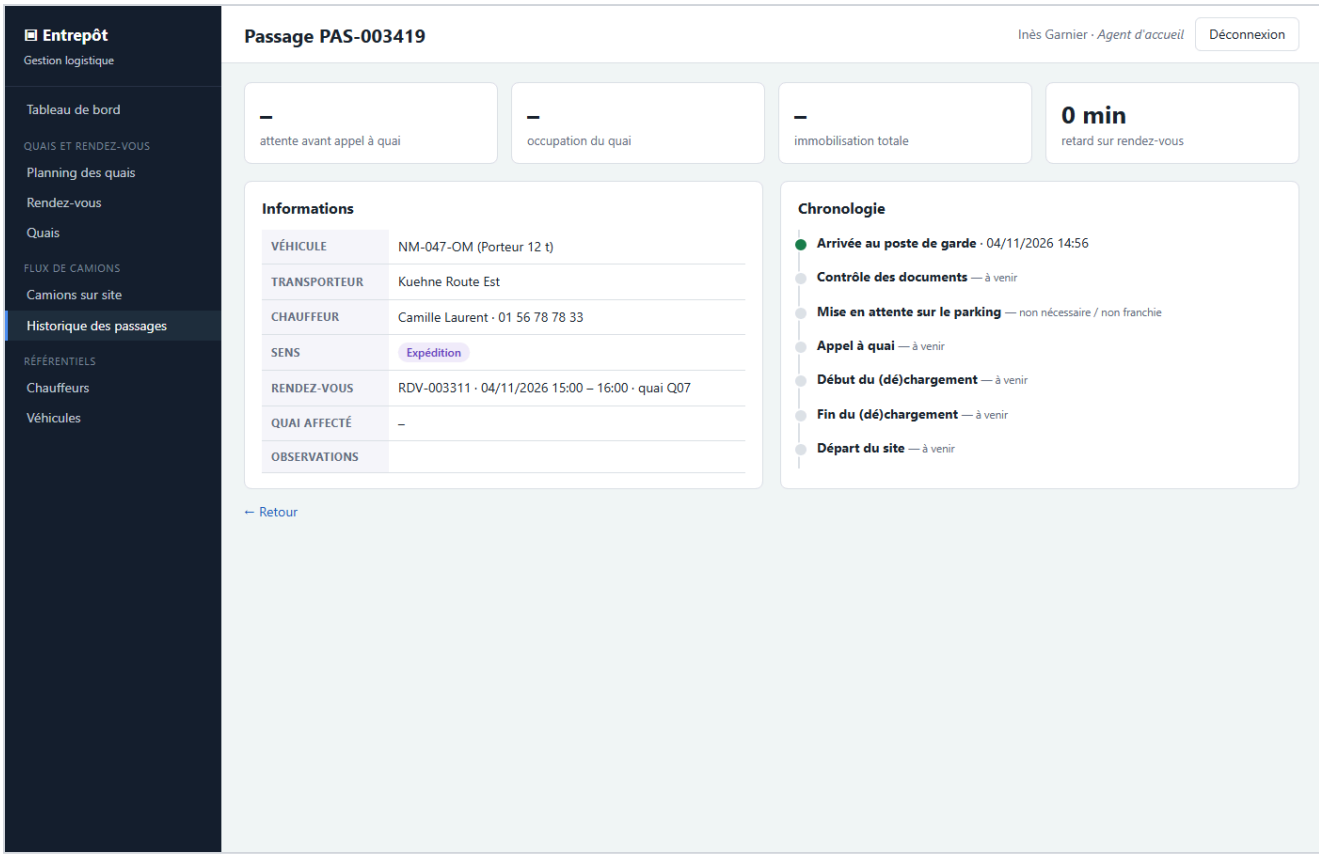


Figure 10 — Détail d'un passage et horodatage de ses étapes

7.5 Réception des marchandises

Objectif. Comparer ce qui est reçu à ce qui était annoncé et tracer les écarts.

Travail réalisé. Avis d'expédition, réception par lots avec dates limites et état des palettes, calcul automatique des écarts à la clôture (constats par article et type d'écart), déclaration manuelle d'un écart, réserves au transporteur, rangement (`avis.php` , `receptions.php` , `reception.php`).

Conclusion. Le calcul des écarts est centralisé dans la procédure `cloturer_reception` . Une livraison non annoncée est gérée : ses articles ressortent en excédent.

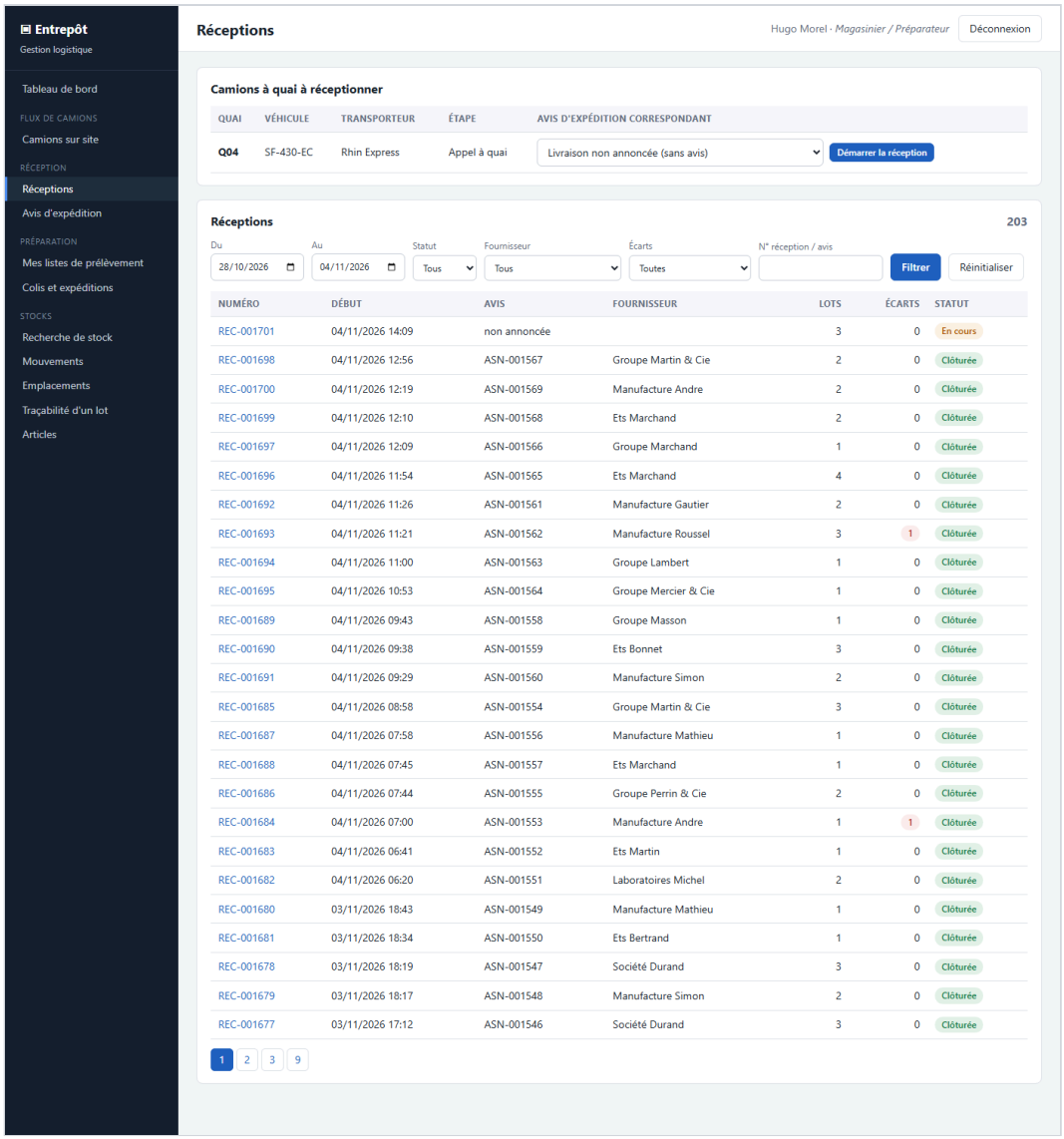


Figure 11 — Liste des réceptions

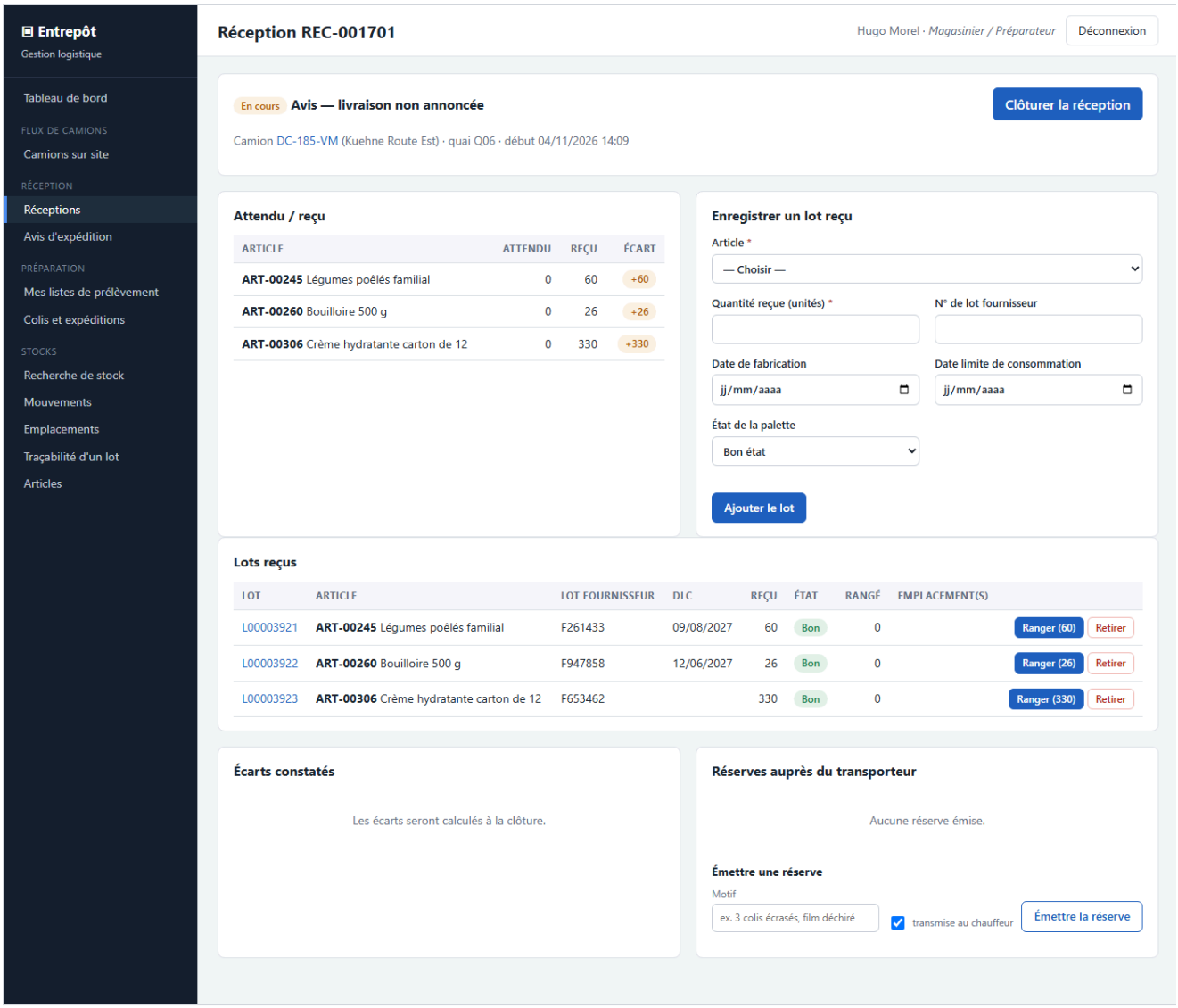


Figure 12 — Saisie d'une réception : attendu / reçu, lots, écarts et réserves

7.6 Emplacements et stocks

Objectif. Savoir où est chaque lot et historiser chaque mouvement.

Travail réalisé. Recherche multicritère et plein texte, export CSV, carte des emplacements, proposition d'emplacements, transferts, ajustements d'inventaire, historique des mouvements (`stock.php`, `emplacements.php`, `mouvement_form.php`, `mouvements.php`).

Conclusion. Le stock n'évolue que par des mouvements, ce qui garantit sa cohérence. Compatibilité produit / zone, blocage et capacité sont contrôlés par la base. Par choix de traçabilité, un mouvement ne se supprime pas : il se corrige par un ajustement.

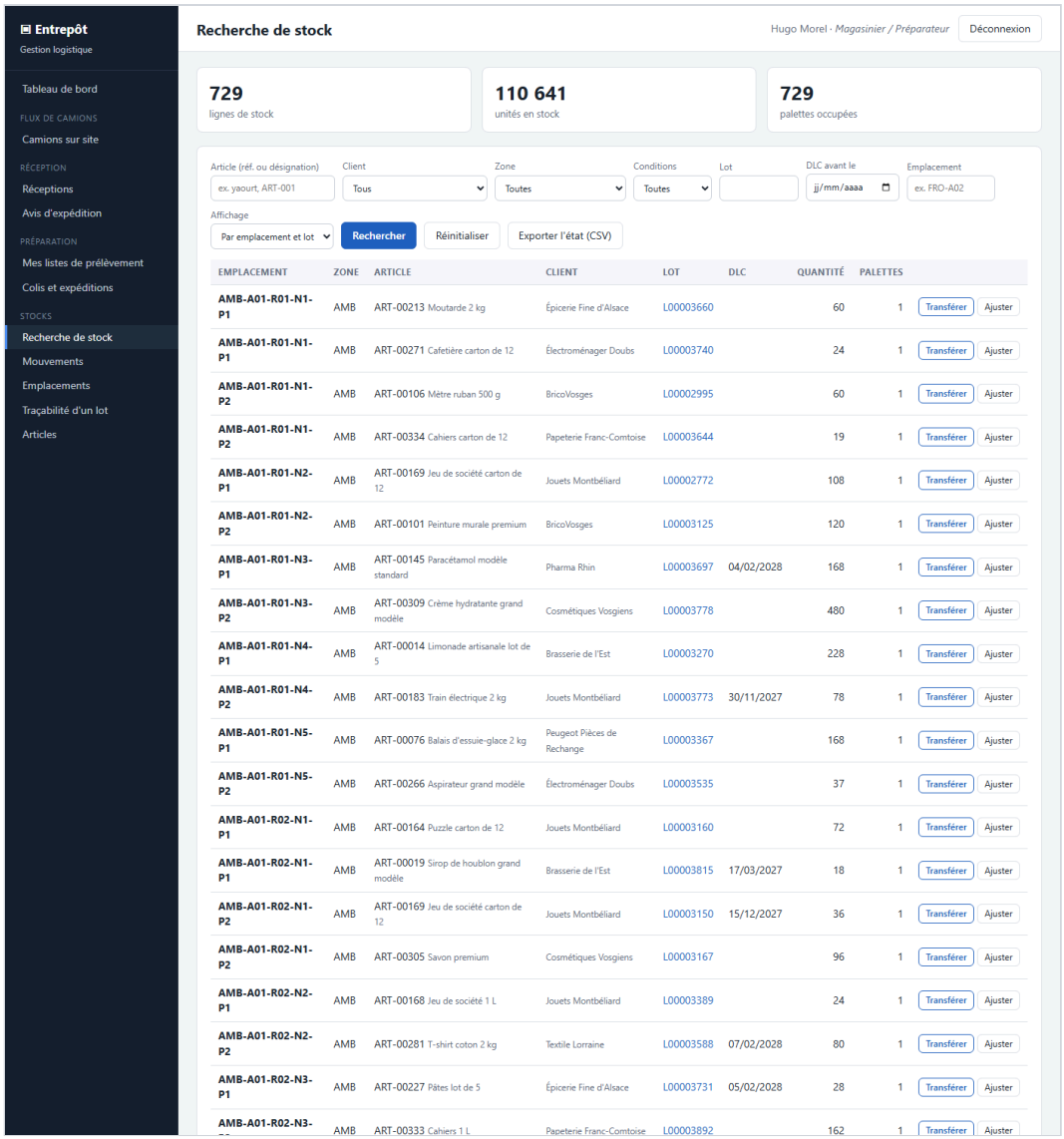


Figure 13 — Recherche de stock multicritère

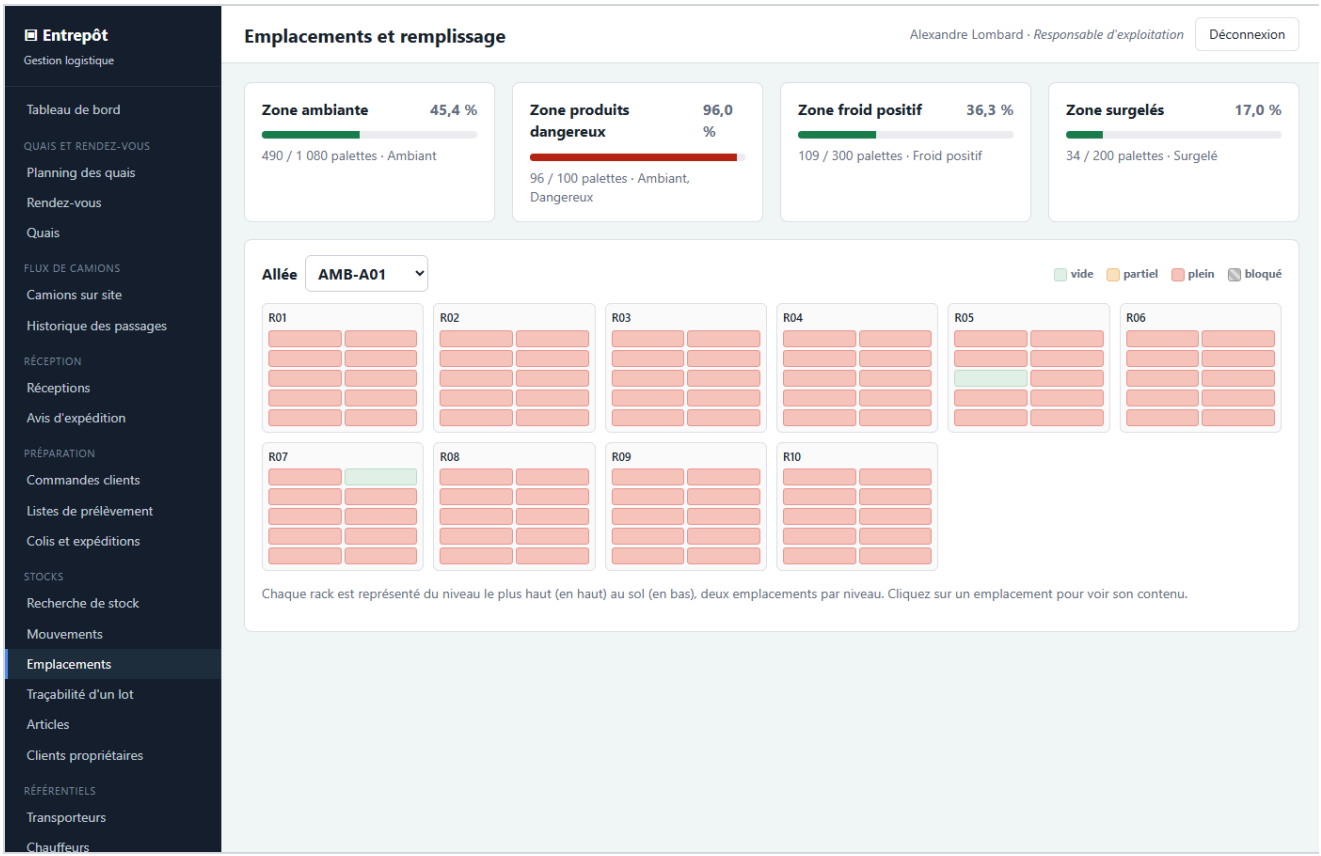


Figure 14 — Carte des emplacements

7.7 Préparation et expédition

Objectif. Sortir les marchandises selon une règle FIFO ou FEFO, jusqu'au chargement du camion.

Travail réalisé. Commandes clients, génération et affectation des listes, prélèvement avec signalement des manquants, colis et palettes avec numéro SSCC, chargement avec contrôle de la capacité du camion, bon de livraison et lettre de voiture imprimables (`commandes.php`, `preparation.php`, `liste.php`, `colisage.php`, `expedition.php`, `document.php`).

Conclusion. La chaîne est complète de la commande au départ du camion, qui fait passer la commande à « expédiée ». La capacité du camion (nombre de palettes et poids maximal) est garantie par deux déclencheurs sur les unités d'expédition : une unité qui ne tient plus reste à quai, et sa commande attend un autre camion. Cette partie n'était pas exigée dans l'interface.

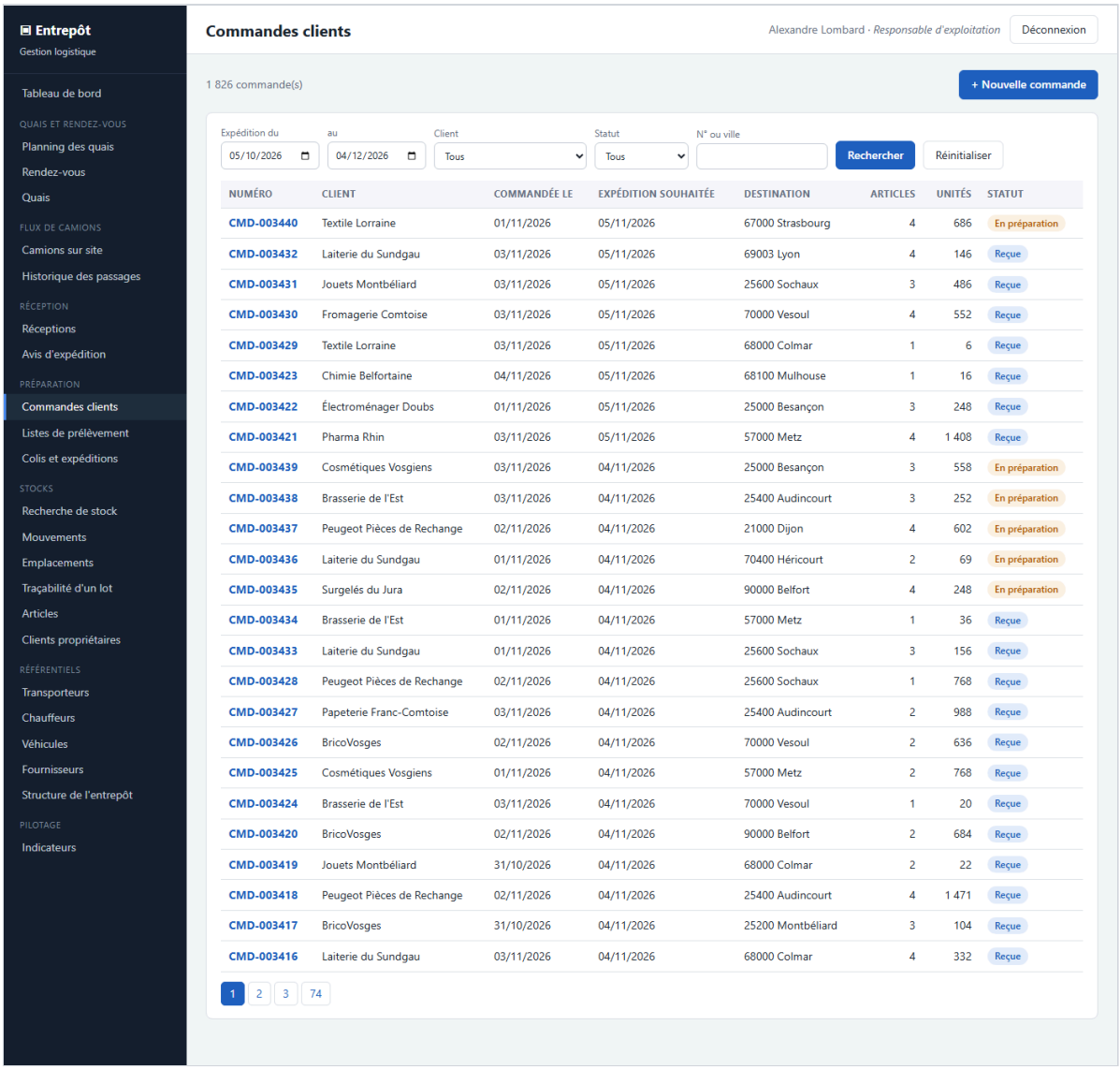


Figure 15 — Commandes clients

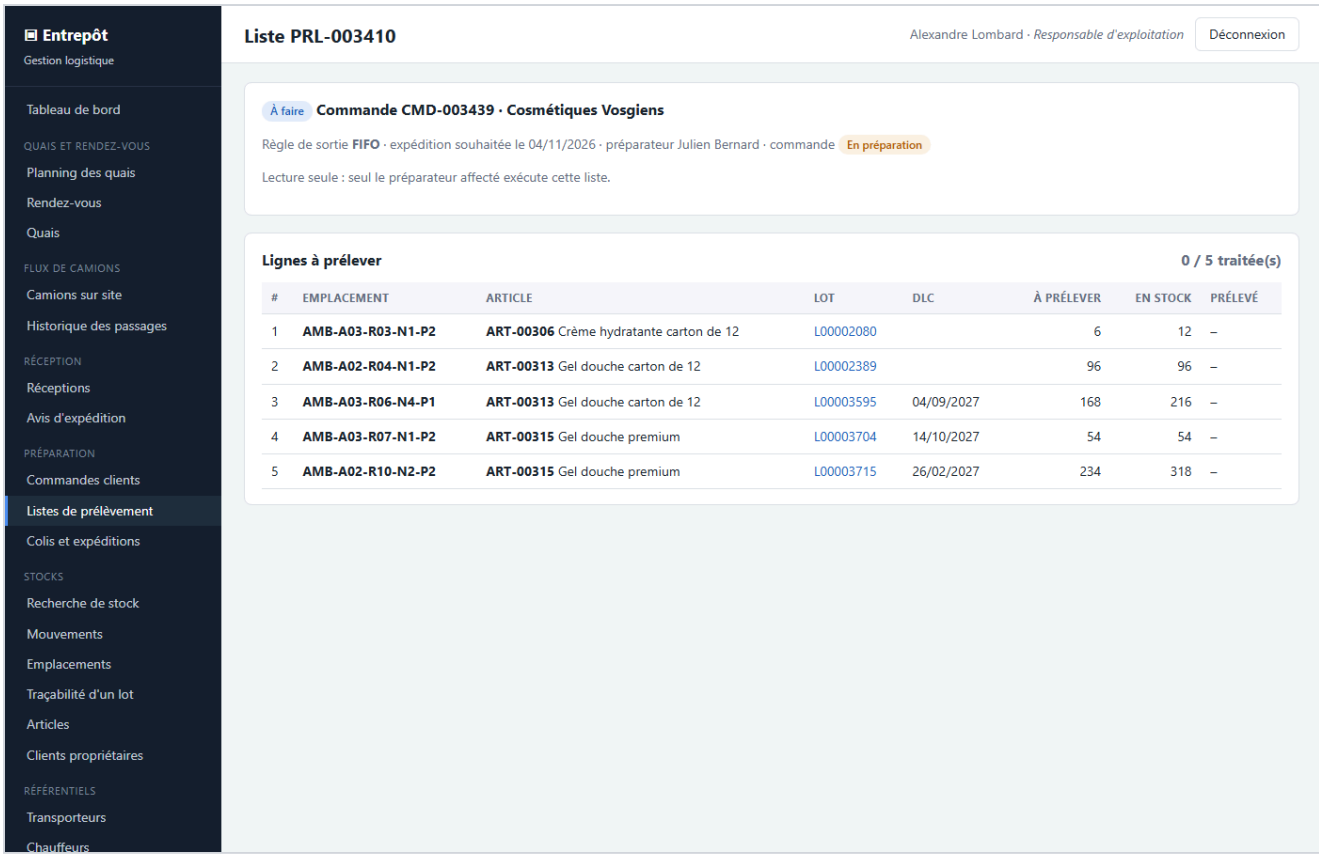


Figure 16 — Liste de prélèvement

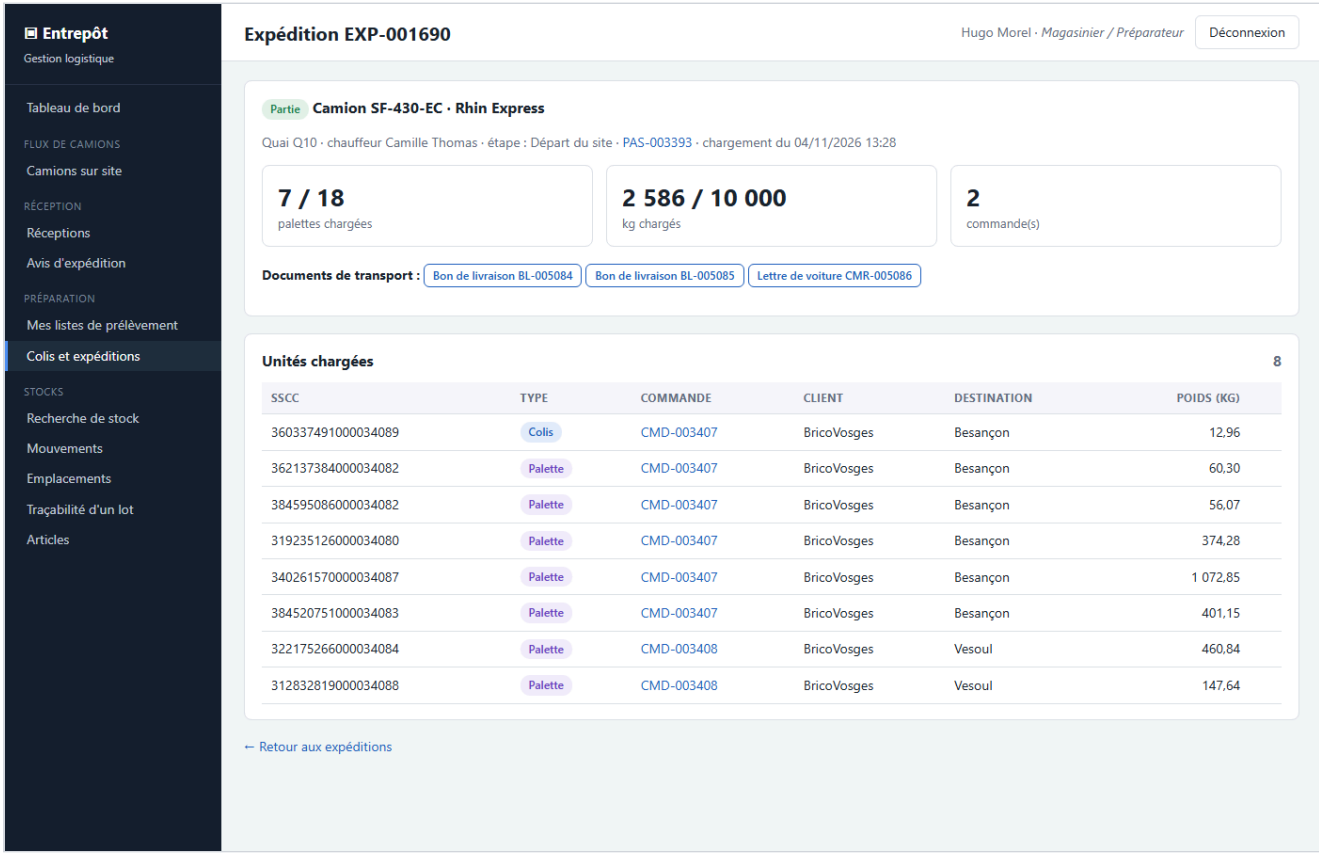


Figure 17 — Chargement d'une expédition et documents de transport

7.8 Traçabilité et indicateurs

Objectif. Retrouver l'historique d'un lot et piloter l'activité.

Travail réalisé. `lot.php` retrace la réception, les emplacements, la commande et le camion. `indicateurs.php` présente sur une période les indicateurs du sujet, en chiffres, tableaux et graphiques, avec un rapport imprimable.

Conclusion. Tous les indicateurs demandés sont produits. Les graphiques fonctionnent sans connexion internet.

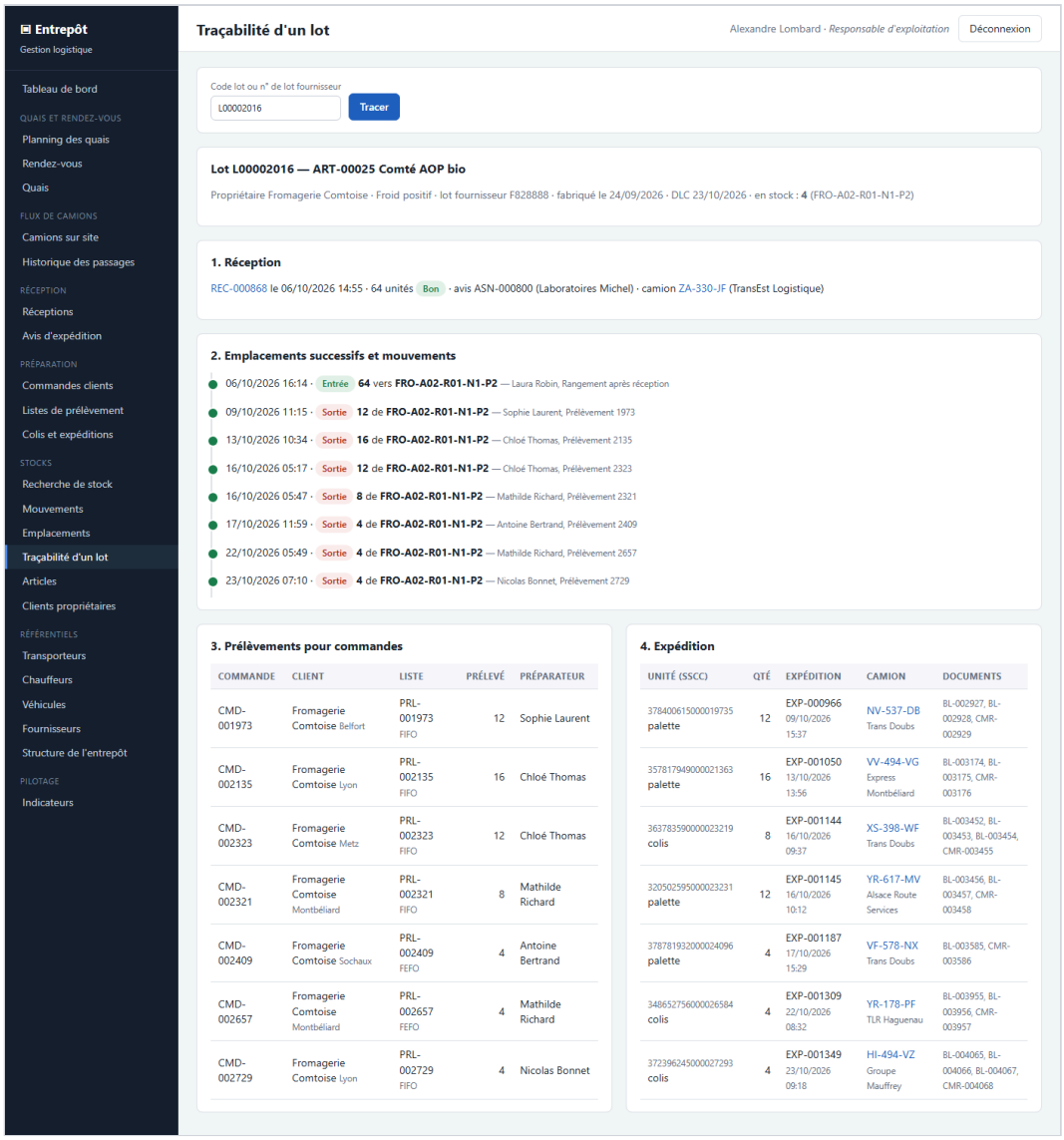


Figure 18 — Traçabilité complète d'un lot

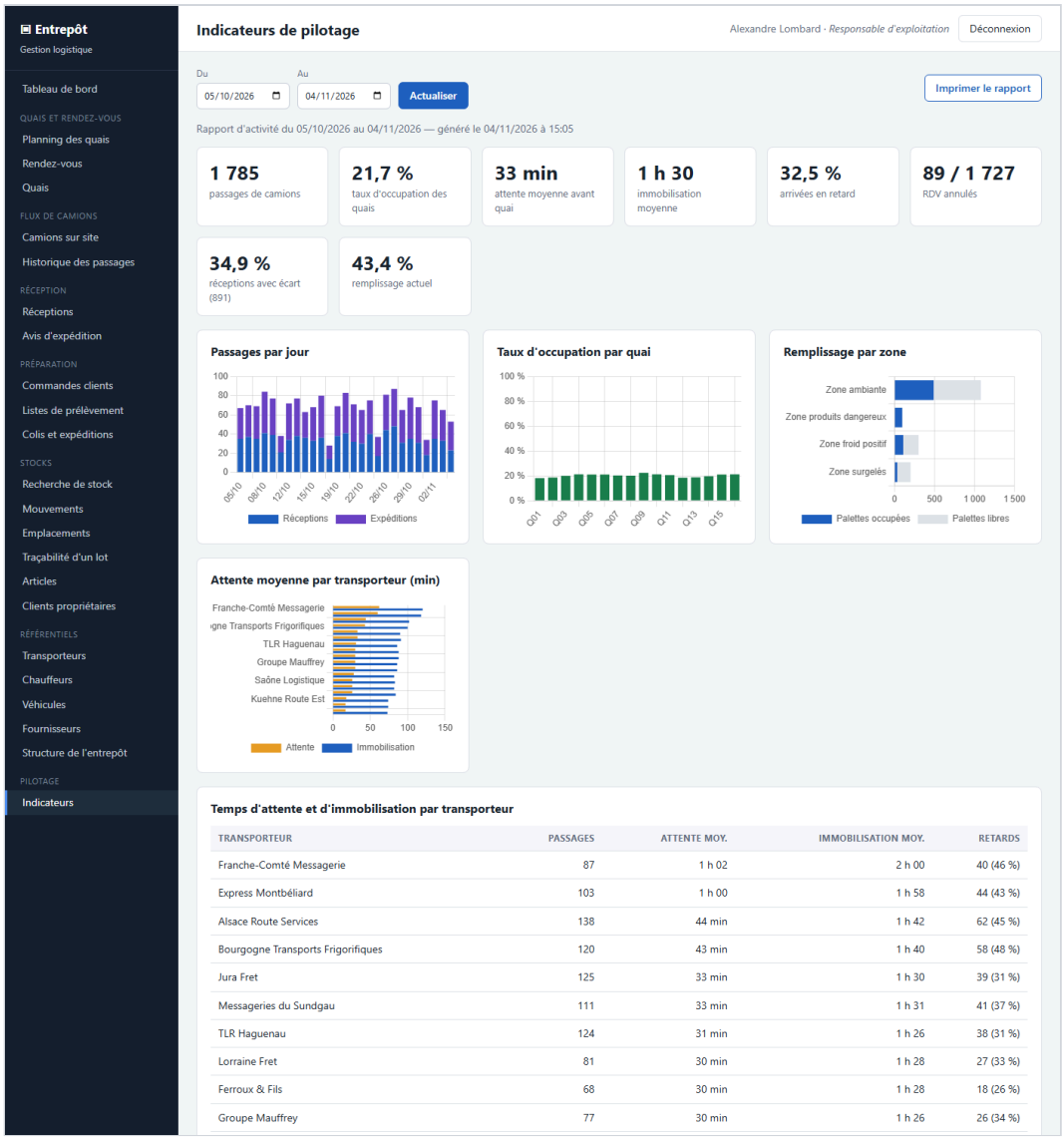


Figure 19 — Indicateurs de pilotage

7.9 Choix de périmètre sur les suppressions

Les données d'historique ne sont pas supprimables depuis l'interface : mouvements de stock, passages, réceptions, expéditions et documents de transport. Les supprimer romprait la traçabilité exigée par le sujet. Ces objets se corrigent, s'annulent ou s'ajustent selon leur nature.

8. Conclusion

Le système couvre l'ensemble du sujet. Les fonctionnalités web minimales sont réalisées, ainsi que la préparation et l'expédition, qui font partie du sujet. Nous n'avons rien ajouté au-delà : le diagramme de cas d'utilisation, le modèle conceptuel, la base et le site décrivent le même périmètre.

Le choix le plus structurant a été de placer les règles métier dans la base : le stock, les quais et les étapes restent cohérents quel que soit le chemin d'écriture. Ce choix a un coût : la logique est répartie entre SQL et PHP, et le schéma dépend de MariaDB à cause des séquences.

Plusieurs améliorations resteraient possibles : la réaffectation automatique des rendez-vous d'un quai hors service, le blocage des comptes après plusieurs échecs de connexion, et un parcours de préparation sur terminal mobile.

Bilan personnel. Ce projet nous a permis de mettre en application l'ensemble du cours sur les bases de données, des requêtes les plus simples jusqu'à l'optimisation. Nous sommes partis de l'analyse, avec les cas d'utilisation puis le modèle entité-association, avant de la traduire en un schéma relationnel en troisième forme normale. Nous avons ensuite écrit des requêtes de plus en plus riches : sélections, jointures et agrégations pour les recherches et les indicateurs, puis vues, fonctions, procédures et déclencheurs pour automatiser les règles métier et l'historisation. Enfin, la mesure des temps d'exécution et la lecture des plans d'exécution (`EXPLAIN`) nous ont montré concrètement ce qu'apporte un index bien choisi. Les remarques reçues sur le modèle conceptuel nous ont aussi appris à vérifier la cohérence entre les modèles, du diagramme de cas d'utilisation jusqu'à la base.